

ACE DIGITAL SCORING OFFICE — COMPLETE PRODUCTION BUILD SPECIFICATION

Build a complete production-ready ACE Digital Scoring Office for Accelerated Christian Education (ACE).

This application is intended for real-world deployment in schools and learning centers.

This is NOT a prototype.

This is NOT a proof of concept.

This is NOT a demonstration application.

This is NOT a learning project.

This is NOT a mockup.

Every component must be implemented as production software.

Generate actual working code.

Generate actual implementations.

Generate actual database schema.

Generate actual business logic.

Generate actual security controls.

Generate actual workflows.

Generate actual integrations.

Generate actual testing.

Generate actual deployment configuration.

Do not generate placeholders.

Do not generate TODO comments.

Do not generate skeleton classes.

Do not generate fake functionality.

Do not simulate features.

Do not stub functionality.

Do not leave incomplete implementations.

Everything must be fully connected and operational.

PRIMARY OBJECTIVE

Create a complete digital ACE scoring office where:

1. Teachers upload official ACE score keys.
2. Score keys are processed and converted into structured answer records.
3. Teachers assign PACEs to students.
4. Students complete physical or PDF PACEs.
5. Students upload completed answer pages.
6. OCR extracts handwritten answers.
7. The system compares extracted answers against official score keys.
8. The scoring engine calculates results.
9. Corrections are automatically generated.
10. Students review corrections.
11. Students upload corrected work.
12. OCR rescans corrected work.
13. Scoring is repeated automatically.
14. Teachers monitor progress.
15. Teachers approve completed PACEs.
16. Administrators manage the entire system.

AI must ONLY perform handwriting recognition and answer extraction.

AI must NEVER determine correctness.

AI must NEVER generate grades.

AI must NEVER decide pass or fail outcomes.

All grading, scoring, correction generation, completion tracking, and reporting must be deterministic PHP software.

TECHNOLOGY STACK

Backend

- PHP 8.2+
- Object-Oriented PHP
- Composer
- PSR-4 Autoloading
- PSR-12 Coding Standards

Frontend

- HTML5
- Tailwind CSS
- Alpine.js
- Vanilla JavaScript

Database

- SQLite ONLY

Realtime

- Ratchet WebSockets

OCR

- Google Document AI

PDF Generation

- Dompdf

Spreadsheet Export

- PhpSpreadsheet

Image Processing

- GD
- Imagick if available

Caching

- File Cache
- SQLite Cache

Queue System

- SQLite Database Queue

Storage

- Local Private Storage

Authentication

- Custom PHP Authentication

Authorization

- RBAC
- Policies
- Permission System

Do not use:

- Laravel
- Symfony
- CodeIgniter
- Yii
- CakePHP
- WordPress
- Microservices
- Kubernetes
- Docker dependencies
- External authentication providers

Everything must run from a single PHP application.

DATABASE REQUIREMENTS

Database Engine

SQLite ONLY

All functionality must operate entirely on SQLite.

Application must not require:

- MySQL
- PostgreSQL
- MariaDB

- SQL Server
- Oracle
- MongoDB
- Redis

Required SQLite Configuration

PRAGMA journal_mode=WAL; PRAGMA foreign_keys=ON; PRAGMA busy_timeout=5000; PRAGMA synchronous=NORMAL; PRAGMA temp_store=MEMORY;

All database access must use PDO.

All queries must use prepared statements.

SQL injection vulnerabilities must be impossible.

All critical writes must use transactions.

All foreign keys must be enforced.

All foreign keys must be indexed.

All search-heavy columns must be indexed.

APPLICATION ARCHITECTURE

Use clean architecture.

Use dependency injection.

Use repository pattern.

Use service layer pattern.

Use event-driven architecture.

Use queue-driven background processing.

Use policy-based authorization.

Use centralized validation.

Use centralized exception handling.

Business logic must never exist inside controllers.

Controllers must orchestrate only.

PROJECT STRUCTURE

/app

/Core

/Auth

/Config

/Controllers

/Database

/Entities

/Repositories

/Services

/OCR

/Scoring

/Corrections

/Notifications

/Reports

/Chat

/Audit

/Policies

/Middleware

/Validators

/Events

/Jobs

/Views

/routes

/storage

/public

/config

/bootstrap

/tests

/vendor

USER ROLES

Student

Teacher

Administrator

STUDENT CAPABILITIES

Login

Logout

Dashboard

View Assignments

View PACE Details

Upload Submission Pages

Upload Correction Pages

View Scores

View Corrections

View Attempt History

Track Progress

View Notifications

Chat With Teachers

Manage Profile

Download Approved Reports

TEACHER CAPABILITIES

Login

Logout

Dashboard

Manage Assigned Students

Assign PACEs

Upload Score Keys

Review OCR Results

Review Low Confidence OCR

Review Submissions

Review Corrections

Leave Feedback

Approve Work

Generate Reports

Chat With Students

Manage Profile

ADMINISTRATOR CAPABILITIES

Manage Users

Manage Students

Manage Teachers

Manage Subjects

Manage PACEs

Manage Score Keys

Manage Settings

Manage Roles

Manage Permissions

View Analytics

View Reports

View Audit Logs

System Monitoring

Backup Management

Full Access

PACE CATALOG

Store

Subject

PACE Number

Title

Description

Status

Version

Created Date

Updated Date

Teachers assign PACEs from catalog.

Teachers never upload PACEs.

Students already possess physical or PDF PACEs.

SCORE KEY MANAGEMENT

Supported Uploads

PDF

PNG

JPG

JPEG

Workflow

Teacher Upload

↓

File Validation

↓

Storage



OCR Processing



Answer Extraction



Teacher Verification



Publication



Version Locking

Every score key must be versioned.

Assignments remain permanently attached to the version used at assignment creation.

Never compare images directly.

Always compare structured answers.

OCR SYSTEM

Provider

Google Document AI

Functions

Handwriting Recognition

Question Detection

Answer Extraction

Confidence Calculation

Store

Question Number

Answer

Confidence

Position Data

Source Image

Submission ID

Low Confidence Threshold

0.80

Anything below threshold enters OCR review.

OCR FAILURE RECOVERY

If OCR request fails:

Retry automatically.

Retry 1

Retry 2

Retry 3

Exponential backoff.

Log failure.

Create audit record.

Notify teacher.

If all retries fail:

Mark submission as OCR Failed.

Allow manual retry.

Allow teacher-assisted answer entry.

Preserve workflow continuity.

ANSWER NORMALIZATION

Before scoring:

Trim whitespace.

Normalize casing.

Remove duplicate spaces.

Remove unnecessary punctuation.

Normalize numbers.

Normalize special characters.

Normalize common OCR mistakes.

Examples

24

24.

24

24

All become:

24

DETERMINISTIC SCORING ENGINE

Components

AnswerNormalizer

AnswerComparator

ScoreCalculator

CorrectionGenerator

ScoringEngine

Functions

Correct Answer Detection

Incorrect Answer Detection

Missing Answer Detection

Percentage Calculation

Confidence Averaging

Correction Generation

Attempt Comparison

Progress Tracking

Store all results permanently.

Never overwrite history.

CORRECTION SYSTEM

Automatically generate correction records.

Students see

Wrong Answer

Correct Answer

Question Number

Correction Status

Completion Percentage

Students upload corrected work.

System rescans automatically.

Repeat until completed.

ATTEMPT HISTORY

Never overwrite submissions.

Every upload creates:

Attempt 1

Attempt 2

Attempt 3

Attempt N

Store complete history forever.

Teachers and administrators can review every attempt.

FILE STORAGE

Private Storage Only

/storage/private

score-keys

submissions

chat-attachments

reports

exports

temp

Files must never be publicly accessible.

Serve files through authorized controllers only.

Use signed download URLs.

FILE VALIDATION

Validate

MIME Type

Extension

Size

Integrity

Corruption

Image Dimensions

PDF Structure

Reject invalid files.

Log every rejection.

REALTIME CHAT

Use Ratchet WebSockets.

Features

Teacher ↔ Student Messaging

Image Attachments

File Attachments

Read Receipts

Typing Indicators

Online Status

Unread Counters

Real-Time Updates

Fallback

If WebSocket unavailable:

Automatically switch to AJAX polling.

Messaging must continue functioning.

NOTIFICATION SYSTEM

Events

Assignment Created

Submission Uploaded

OCR Completed

OCR Failed

OCR Review Required

Scoring Completed

Corrections Required

Teacher Feedback

New Message

PACE Completed

System Alerts

Features

Unread Count

Read Status

History

Filtering

Realtime Updates

Fallback Polling

REPORTING SYSTEM

Teacher Reports

Student Progress

PACE Progress

Completion Rates

Scores

Corrections

Assignment Status

Admin Reports

Platform Usage

Teacher Activity

Student Activity

OCR Statistics

Performance Metrics

Exports

PDF

CSV

Excel

AUDIT LOGGING

Track every significant action.

Examples

Login

Logout

Upload

Delete

Assign

Approve

Reject

Review

Update

Create

Download

Settings Changes

User Management

Audit logs must be immutable.

Audit logs must never be editable.

BACKUP SYSTEM

Automatic Daily Backups

Automatic Weekly Backups

Automatic Monthly Backups

Backup Verification

Backup Logging

Restore Validation

Retention Policies

Backup Failure Alerts

Administrator Notifications

ERROR HANDLING

Centralized exception handling.

User-friendly messages.

Detailed logs.

No stack traces exposed publicly.

Structured error responses.

Automatic recovery where possible.

SECURITY

Implement

RBAC

Policies

CSRF Protection

Secure Sessions

Argon2id Password Hashing

Rate Limiting

Input Validation

Output Escaping

XSS Protection

SQL Injection Protection

Clickjacking Protection

Content Security Policy

HTTPS Enforcement

Secure Cookies

Signed URLs

Audit Logging

Session Regeneration

Brute Force Protection

Account Lockout Thresholds

Password Reset Security

File Upload Security

Authorization Checks On Every Request

Students access only their records.

Teachers access only assigned students.

Administrators access everything.

No route may bypass authorization.

TESTING

Generate

Unit Tests

Integration Tests

Authentication Tests

Authorization Tests

OCR Tests

Scoring Tests

Correction Tests

Report Tests

File Upload Tests

Security Tests

Database Tests

Queue Tests

WebSocket Tests

End-To-End Tests

FINAL REQUIREMENT

Generate a complete production-ready ACE Digital Scoring Office built entirely with:

PHP 8.2+

SQLite

Tailwind CSS

Alpine.js

JavaScript

Ratchet WebSockets

Google Document AI

The application must be secure, maintainable, scalable, fault-tolerant, fully implemented, fully connected, deployable, and suitable for real-world school usage without requiring major architectural changes.